Intelligent Systems

Knowledge & reasoning

Imperative vs. declarative languages

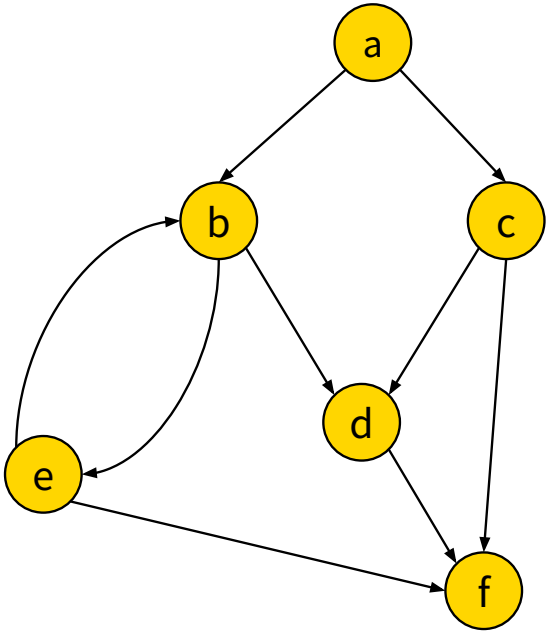
Imperative languages:

- Java, C++, Python, …
- Knowledge: data structures; Reasoning: algorithms - control flow and computation rules.
- Focus is on how to do something.

Declarative languages:

- Prolog, rule-based systems.
- Knowledge and reasoning are mixed: the system reasons automatically over the knowledge base.
- Focus is on what something is like.

Imperative vs. declarative example



Task: Find a path from **a** to **f** of length 3.

Imperative programming:

1. Start in S = a.
2. For all children C = C_0 to C_n of S:
   - 2.1. Transition from S to C.
   - 2.2. For all children of C …
   - 2.3. …

Declarative programming:

The solution looks like this:

a ⟶ X ⟶ Y ⟶ f

Find X and Y.

Prolog (PROgraming in LOGic)

- A declarative language based on first-order logic.
- Developed in 1972 by Alain Colmerauer with Philippe Roussel.
- Unification for pattern matching.
- Inference engine based on depth-first-search.
- Backtracking to explore alternate unifications.
- Tools to refine the search - cut (!), fail.

Propositional logic in Prolog

Logical operators: AND (,) OR (;) NOT (\+).

Example:

```
% --- Rules (logical implications) ---
go_outside :- sunny, warm.          % (sunny ∧ warm) → go_outside
take_umbrella :- rainy ; cloudy.    % (rainy ∨ cloudy) → take_umbrella
happy :- go_outside.                % go_outside → happy

% --- Facts (atomic propositions) ---
sunny.   % It is sunny.
warm.    % It is warm.
```

We ask Prolog:

```
?- happy.   % Are we happy?

true.
```

First-order logic in Prolog

First-order logic extends propositional logic with:

- Variables (e.g., X, Y, Z, Element, Person, …)
- Predicates (specifying properties or relations).
- Quantifiers:
  - ∀ (for all) - universal
  - ∃ (there is) - existential
- Enables reasoning about **objects** (e.g., a, b, c, element, person, …)

Stating facts about objects

- Objects (beginning with small letters):

  ```
  car, ball, sun, peter, teacher, ...
  ```

- Variables (Beginning with capital letters):

  ```
  X, Y, Z, Person, Element, ...
  ```

- Properties:

  ```
  property(object)
  ```

- Relations:

  ```
  relation(object1, object2, ...)
  ```

Stating facts about objects - examples

- John and Peter are males. Hannah is a female.

```
male(john).
male(peter).
female(hannah).
```

- John and peter are siblings.

```
siblings(john, peter).
```

- Peter is Hannah’s parent.

```
parent(peter, hannah).
```

- Peter is 35 years old. Hannah is 12 years old.

```
age(peter, 35).
age(hannah, 12).
```

Quantifiers ∃ and ∀

Existential quantifier ∃

$$\exists\ X\text{: property\_a}(X) \wedge \text{property\_b}(X), \ldots$$

```
property_a(X), property_b(X), ...
```

Universal quantifier ∀

$$\forall\ X\text{: property\_a}(X) \rightarrow \text{property\_b}(X), \ldots$$

property_b(X) :- property_a(X), …

Quantifiers ∃ and ∀ - examples

- There is a male. (∃ X: male(X))
```
male(X). % Who is a male?
male(_). % Is there a male?
```
- Peter has a child. (∃ X: parent(peter, X))
```
parent(peter, X). % Who is Peter's child?
parent(peter, _). % Is Peter a parent?
```
- John and Peter are brothers.
```
siblings(john, peter), male(john), male(peter).
```
- Every male sibling is a brother (∀ X: male(X), sibling(Y, X) → brother(X).
```
brother(X) :- male(X), sibling(_, X).
```
- If you are my sibling then I am also your sibling (symmetric relation).
```
sibling(X, Y) :- sibling(Y, X).
```

Non-determinism of Prolog

- In imperative programming languages, a variable can be assigned only one value at a time.
- In Prolog, a variable can be assigned multiple values (non-determinism).

```
small(ball). small(dice). small(coin).
round(ball). round(moon). round(coin).

?- small(X), round(X).
         |         |
         X = ball  X = ball
         X = dice  X = coin
         X = coin
```

Prolog answers:

```
X = ball
X = coin
```

findall, setof

Sometimes, we want all solutions to be assembled in a list.

```
findall(Object, Goal, List)
```

If we want to eliminate duplicates:

```
setof(Object, Goal, List)
```

Example:

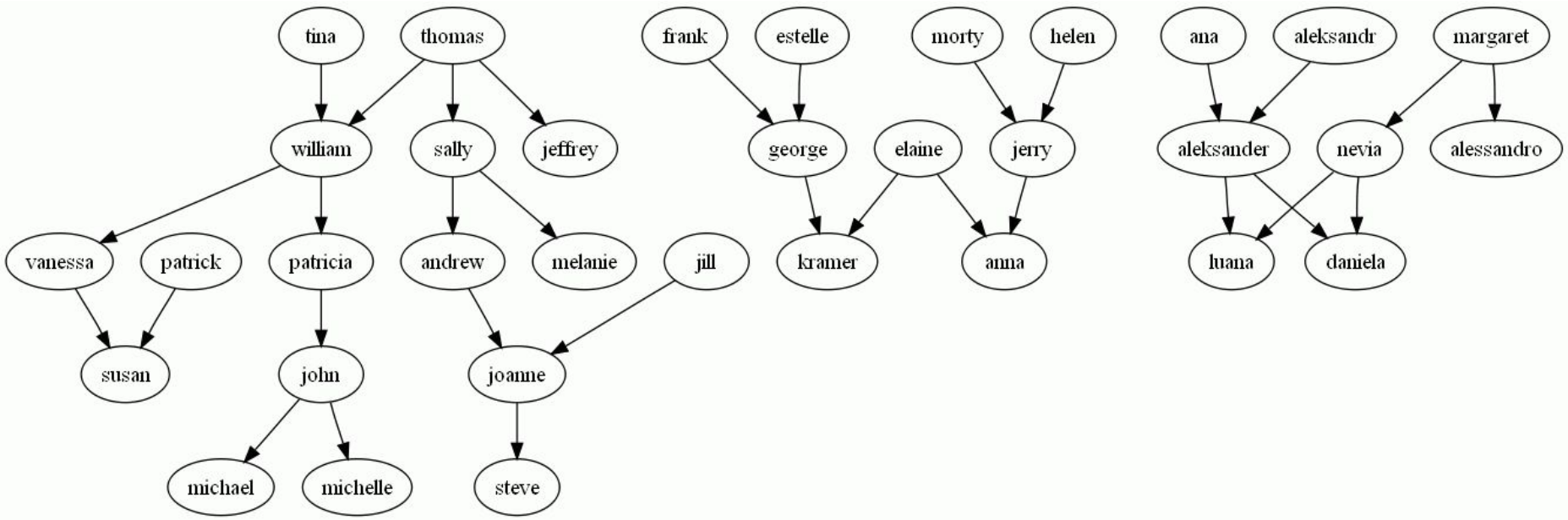
```
?- findall(X, (small(X), round(X)), L).
L = [ball, coin]
```

Structures and unification

- Operator ‘=’ tries to unify two expressions.
  - `X = apple` (succeeds, variable X is assigned object ‘apple’).
  - `X = apple, X = banana` (fails, objects ‘apple’ and ‘banana’ cannot be unified).
  - `X = Y, X = apple` (succeeds, X and Y are both assigned object ‘apple’).
  - `X/Y = apple/banana` (succeeds, X =’apple’ and Y=’banana’).
  - `one+two-Z = X+Y-three` (succeeds, X =’one’, Y=’two’, Z=’three’).
- Operator ‘\=’ succeeds if the expressions cannot be unified.
  - `apple \= banana` (succeeds, ‘apple’ is not ‘banana’).
  - `X \= Y` (fails, variables X and Y can be unified).
  - `X = apple, X \= banana` (succeeds, X is ‘apple’ and it is not ‘banana’).
  - X/X \= apple/banana (succeeds, X cannot be ‘apple’ and ‘banana’).

Exercise

Load the Prolog database *family_tree.pl*, containing the below family tree.



Exercise

Ask the following questions in Prolog:

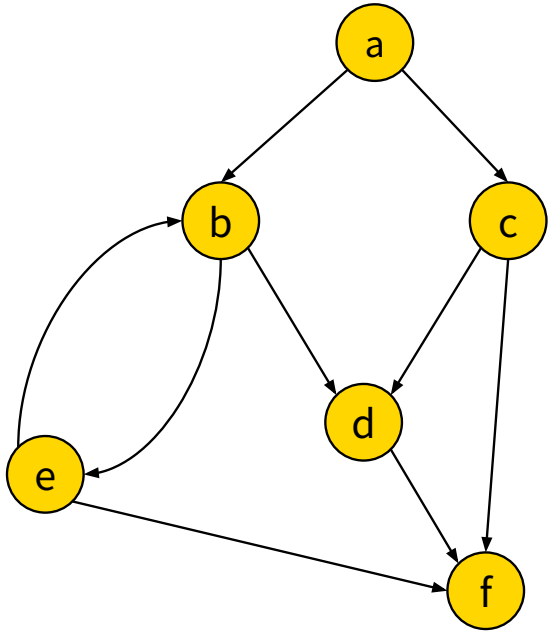
- Is William male?
- Is Thomas female?
- Who is male?
- Are there any females?
- Who are Elaine's children?
- Who are Jeffrey's siblings?
- Who are Jeffrey's true siblings?
- Who are Jeffrey's brothers?

Exercise

Define new predicates:

- aunt(X, Y) - X aunt of Y.
- cousin(X, Y) - X is cousin of Y.
- parent(X) - X is parent.
- daughter(X, Y) - Y is daughter of X.
- daughters(X, L) - L are all daughters of X.

Example



Find a path from **a** to **f** of length 3.

```
link(a, b). link(a, c). link(b, d). link(c, d). link(b, e).
link(e, b). link(c, f). link(e, f). link(d, f).

?- link(a, X), link(X, Y), link(Y, f).
```

Define a predicate **path** from A to B of any length (recursively).

```
path(A, B) :- link(A, B).
path(A, B) :- link(A, X), path(X, B).
```

Is there a path from **e** to **d**?

```
?- path(e, d).
```

Is there a path from **d** to **e**?

```
?- path(d, e).
```

Arithmetics

- Unification operator ‘=’ cannot be used for arithmetic operations.

```
X = 1 + 2.
```

  Assigns the structure ‘obj1 + obj2’ to variable X, not the result of addition.

- Symbols +, -, *, / can be used to define data structures.
- Arithmetic result is assigned with the ‘is’ operator:

```
X is 1 + 2.
```

Lists

- A list is written using [ ]

```
L = [one, two, three], L = [1, 2, 3, 4, 5].
```

- A list can contain constants, variables, other lists

```
L = [12, a, b, c, X, [1, 2, 3, b]].
```

- We can separate the head from the tail
  - `[a, b, c] = [a | [b, c]]`
  - `[a, b, c] = [a, b | [c]]`
  - `[a, b, c] = [a, b, c | []]`

Exercise

- Remember: a head is unified with an object, the tail is unified with a list.
- Determine the values of variables for:
  - [H|T] = [a, b, c]
  - [H|T] = []
  - [H1, H2|T] = [a, b, c]
  - [H1, H2|T] = [a, b]
  - [H1, H2|T] = [a]

Examples

A predicate `add/3` that adds an element to a list to the front.

```
add(E, List, [E|List]).
```

A predicate `delete/2` that removes the first element from the list.

```
delete([], []).
delete([_|Tail], Tail).
```

Exercises

- Define a predicate `count/2` that relates a list to its size.
- Define a predicate `add_end/3` that adds an element to the end of the list.
- Define a predicate `insert/4` that inserts an element to the i-th place in the list.
- Define a predicate `remove/3` that removes the i-th elements from the list.
- Define a predicate `reverse/2` that reverses a list.
- Define a predicate `dup(L1, L2)`, where elements of L2 are duplicated elements of L1, e.g.: `L1 =[x, y, z],L2 = [x, x, y, y, z, z].`
- Define a predicate `increasing(L)`, which is true if the numbers in L are sorted in the ascending order.
- define a predicate `set(L)`, which is true, if L is a set (contains no duplicates).

Challenge

Solve the Missionaries and Cannibals problem in Prolog.

Image source: https://missionaries-and-cannibals.soft112.com/